Proyecto Final #1

Segmentación Semántica Multiclase con Transfer Learning usando SAM2

Universidad San Francisco de Quito
Maestría en Inteligencia Artificial
Visión Artificial 202610

Pablo Encalada

Septiembre 2026

1. Introducción

La segmentación semántica consiste en asignarle una clase a cada píxel de una imagen, lo cual es fundamental en aplicaciones como conducción autónoma, imágenes médicas y análisis de escenas. En este proyecto se aborda este problema en el contexto de SuperTuxKart, un videojuego de carreras arcade 3D open-source, donde el objetivo es que un modelo pueda recibir un frame del juego y clasificar cada píxel en una de 7 categorías: Background, Track, Kart, Pickup, Nitro, Bomb y Projectile.

La idea central del proyecto es aprovechar Transfer Learning con SAM2 (Segment Anything Model 2) de Meta, un modelo que ya fue entrenado para entender estructura visual en millones de imágenes. En vez de entrenar una red completa desde cero, se congela el encoder de SAM2 y se entrena únicamente un decoder liviano que adapta esas representaciones visuales a nuestro problema específico de 7 clases. Esto reduce drásticamente el tiempo de entrenamiento y la cantidad de datos necesarios para obtener buenos resultados.

El dataset proviene de pystk, una interfaz de Python que interactúa directamente con el motor gráfico del juego, lo que permite generar máscaras de segmentación precisas a nivel de píxel. Sin embargo, el dataset presenta un desbalanceo severo entre clases, lo cual representó uno de los principales desafíos del proyecto.

2. Metodología

2.1. Dataset

El dataset contiene 1500 pares de imágenes y máscaras organizados en dos carpetas: `images/` con los frames originales de 400x400 píxeles en RGB, y `masks/` con las máscaras

de segmentación donde cada píxel tiene un valor entero entre 0 y 6 que representa su clase. La relación entre imagen y máscara se establece por nombre (por ejemplo, `abyss_frame_0128.png` corresponde a `abyss_mask_combined_0128.png`).

Las 7 clases y sus valores son:

Clase	Valor
Background	0
Track	1
Kart	2
Pickup	3
Nitro	4
Bomb	5
Projectile	6

Cuadro 1: Clases del dataset y sus valores en la máscara.

El dataset presenta un desbalanceo muy grande: Background y Track juntos representan más del 90 % de los píxeles, mientras que Nitro, Bomb y Projectile son menos del 0.5 %. Además, Projectile aparece en prácticamente solo 2 frames de los 1500. La Figura 1 muestra la distribución de píxeles y frames por clase.

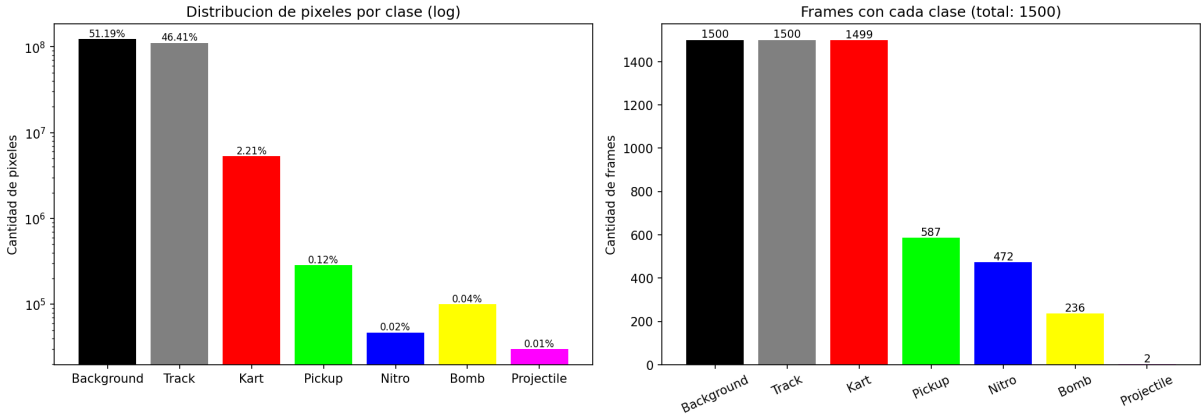


Figura 1: Distribución de píxeles por clase (escala logarítmica) y cantidad de frames que contienen cada clase.

2.2. Preprocesamiento y Data Augmentation

Las imágenes se redimensionan a 512x512 píxeles ya que SAM2 trabaja mejor con resoluciones más altas. Las máscaras se redimensionan con interpolación nearest para no generar valores intermedios entre clases. Se aplica normalización ImageNet (media y desviación estándar) ya que el encoder de SAM2 fue pre-entrenado con esa normalización.

Para data augmentation en entrenamiento se aplican flips horizontales, rotación leve (± 15 grados) y cambios aleatorios de brillo, contraste y saturación. Estas transformaciones se

aplican tanto a la imagen como a la máscara (en el caso de transformaciones geométricas) para mantener la correspondencia.

2.3. Manejo del Desbalance de Clases

Para combatir el desbalance se implementaron tres estrategias:

Oversampling con WeightedRandomSampler: los frames que contienen clases raras (Nitro, Bomb, Projectile) reciben 5 veces más probabilidad de ser seleccionados durante el entrenamiento. De los 1200 frames de train, 536 contienen clases raras.

Pesos en CrossEntropyLoss: se asignan pesos inversamente proporcionales a la frecuencia de cada clase, con Background en 0.5 y Projectile en 15.0, para que los errores en clases raras penalicen más al modelo.

Dice Loss: se combina con CrossEntropyLoss (30 % CE + 70 % Dice) para medir directamente el overlap entre predicción y ground truth por clase, lo cual beneficia especialmente a las clases con pocos píxeles.

2.4. Arquitectura del Modelo

La arquitectura sigue el paradigma de Transfer Learning con dos componentes:

El encoder es el vision encoder de SAM2 Hiera-Tiny, con 27.2 millones de parámetros, todos congelados. Este backbone pre-entrenado extrae features visuales de alta calidad y produce 3 niveles de features multiescala vía FPN (Feature Pyramid Network) de 256 canales cada uno a resoluciones de 128x128, 64x64 y 32x32 píxeles.

El decoder es una red ligera con 2.1 millones de parámetros entrenables. Toma los 3 niveles FPN, los procesa con convoluciones 3x3, los lleva todos a la misma resolución mediante interpolación bilinear y los concatena (384 canales). Luego fusiona con dos capas convolucionales con BatchNorm, ReLU y Dropout (0.1), y produce la clasificación pixel a pixel con un Conv 1x1 de 7 canales. Al final se hace un upsample bilinear para volver a la resolución original de 512x512.

En total solo se entrenan 2.1M de 29.3M parámetros (aproximadamente el 7 %), que es la esencia del transfer learning.

2.5. Entrenamiento

Se utiliza un split 80/20 para train/validación con seed fija para reproducibilidad. El batch size es de 4 debido a las restricciones de memoria del encoder de SAM2 procesando imágenes de 512x512. El optimizador es Adam con learning rate inicial de 5×10^{-4} aplicado exclusivamente a los parámetros del decoder. Se utiliza un scheduler ReduceLROnPlateau que reduce el learning rate a la mitad si el validation loss no mejora en 5 epochs. El entrenamiento se ejecuta por 60 epochs y se guarda el mejor modelo según el mIoU en validación.

2.6. Métrica de Evaluación

Se utiliza mean Intersection over Union (mIoU) como métrica principal. El IoU por clase se calcula como:

$$IoU = \frac{|Predicción \cap GroundTruth|}{|Predicción \cup GroundTruth|} \tag{1}$$

El mIoU es el promedio de los IoU de todas las clases presentes en el conjunto de validación.

3. Desarrollo

El entrenamiento convergió bien, con el loss bajando de manera consistente tanto en train como en validación sin signos de overfitting. El mIoU subió de aproximadamente 0.50 en el epoch 1 hasta 0.8064 en el epoch 58, donde se guardó el mejor modelo.

La Figura 2 muestra las curvas de entrenamiento. Entre los epochs 14 y 46 el mIoU osciló entre 0.72 y 0.78 sin converger del todo. Esto ocurre porque el learning rate de 5×10^{-4} es un poco grande para esa etapa del entrenamiento y el optimizador salta alrededor del mínimo en vez de acercarse suavemente. Cuando el ReduceLROnPlateau redujo el LR a $2{,}5 \times 10^{-4}$ en el epoch 47, el modelo se estabilizó y subió a 0.79-0.80. Una segunda reducción a $1{,}25 \times 10^{-4}$ en el epoch 55 le permitió llegar al pico de 0.8064.

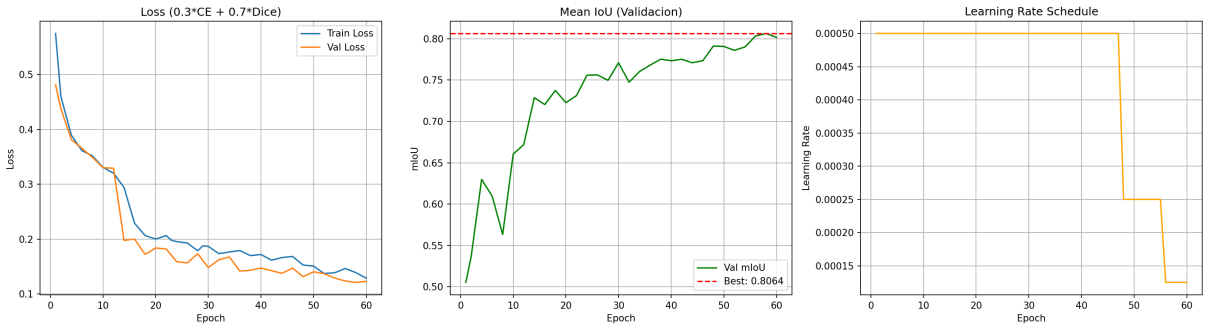


Figura 2: Curvas de entrenamiento: loss (train y validación), mIoU en validación, y learning rate a lo largo de los 60 epochs.

Los resultados finales de IoU por clase evaluados sobre el conjunto de validación son:

Clase	IoU
Background	0.9102
Track	0.9044
Kart	0.9156
Pickup	0.8594
Nitro	0.8037
Bomb	0.7269
Projectile	0.0000
mIoU	0.8534

Cuadro 2: IoU por clase y mIoU sobre el conjunto de validación.

La Figura 3 muestra ejemplos de predicciones del modelo comparadas con el ground truth, y la Figura 4 muestra un ejemplo individual con su overlay.



Figura 3: Predicciones del modelo vs ground truth en muestras aleatorias del conjunto de validación.

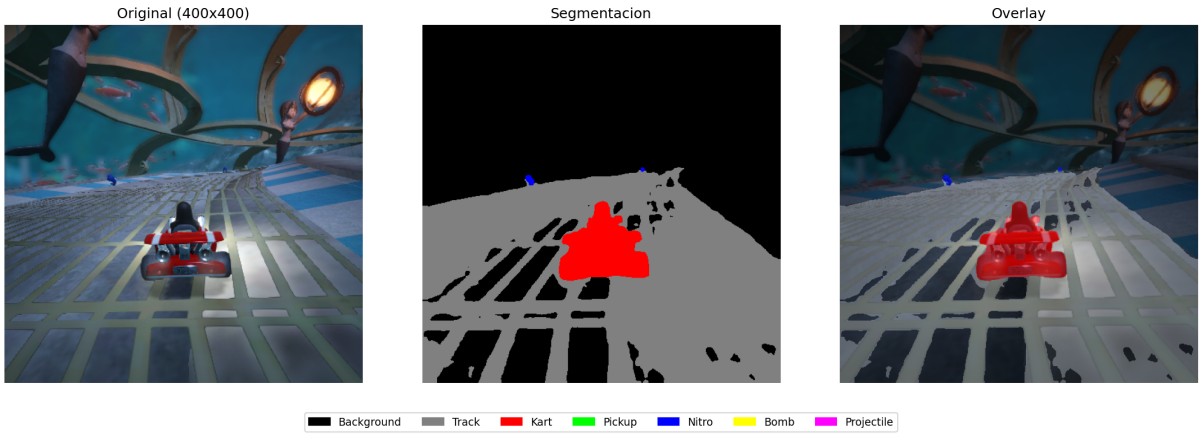


Figura 4: Ejemplo de segmentación: imagen original, mapa de segmentación predicho y overlay.

Background, Track y Kart logran IoU superiores a 0.90, lo cual tiene sentido porque son las clases con más píxeles y objetos de mayor tamaño. Kart en particular obtiene el mejor resultado individual con 0.9156, ya que son objetos grandes con forma bien definida y el encoder de SAM2 ya viene entrenado para detectar objetos. Pickup con 0.8594 y Nitro con 0.8037 son resultados sólidos considerando que son objetos pequeños. Bomb con 0.7269 fue la clase más difícil por la poca cantidad de samples y el tamaño reducido de los objetos. Projectile tiene IoU de 0.0000 porque prácticamente no existe en el dataset.

4. Dificultades Enfrentadas

La principal dificultad fue el desbalanceo extremo del dataset. Background y Track dominan con más del 90 % de los píxeles, lo cual hacía que sin técnicas de compensación el modelo predijera solo estas dos clases. Se necesitó implementar oversampling, pesos en la función de pérdida y Dice Loss para lograr que el modelo también aprenda las clases minoritarias.

La clase Projectile representó un caso imposible de resolver con las técnicas disponibles. Con solo 2 frames de 1500 que la contienen, no hay suficientes ejemplos para que el modelo aprenda a reconocerla independientemente de cuántas técnicas de balanceo se apliquen.

El consumo de memoria del encoder de SAM2 limitó el batch size a 4, ya que procesar imágenes de 512x512 a través del vision encoder de Hiera-Tiny requiere bastante memoria. Esto hizo que el entrenamiento fuera más lento y con más ruido en las métricas por batch comparado con batch sizes más grandes.

La oscilación del mIoU durante el entrenamiento entre los epochs 14 y 46 fue otra dificultad. El learning rate inicial de 5×10^{-4} resultó ser demasiado grande para la convergencia fina, y fue necesario el scheduler ReduceLROnPlateau para reducirlo automáticamente y permitir que el modelo convergiera de manera más estable.

5. Conclusiones

El transfer learning con SAM2 demostró ser más eficiente que entrenar un modelo desde cero. Con solo 2.1M de parámetros entrenables (7 % del total) y 60 epochs se logró un mIoU de 0.8534, superando el 0.83 que se obtuvo con U-Net en la Actividad 2 que tenía todos sus parámetros entrenables y necesitó 80 epochs. Esto se debe a que el encoder de SAM2 ya viene con features visuales de alta calidad aprendidas de millones de imágenes, lo cual le da una ventaja enorme al modelo desde el primer epoch.

La combinación de Dice Loss con CrossEntropyLoss junto con el oversampling fue clave para manejar el desbalance de clases. Sin estas estrategias el modelo se limitaría a predecir Background y Track e ignoraría las clases pequeñas. Con estas técnicas se logró que incluso Nitro y Bomb, que son objetos muy pequeños y poco frecuentes, alcancen IoU de 0.80 y 0.73 respectivamente.

La clase Projectile es imposible de aprender con este dataset ya que aparece en prácticamente 2 frames de 1500, ninguna técnica de balanceo puede compensar esa falta de datos. Para mejorar esto se necesitaría generar más datos con pystk que contengan projectiles, o aplicar técnicas de data augmentation específicas como copy-paste augmentation para insertar projectiles artificialmente en otros frames.